



## 「2025 IA x AI 해커톤」

# 개발 완료 보고서

팀 명 : BookToss

프로젝트명 : BookToss

### ※ 유의사항

1. 본 보고서의 내용은 최대 2 page이내로 작성 (본 표지 제외)
2. 보고서의 설명을 보충하기 위해 필요한 사진 또는 그래프 첨부 가능
3. 제출 서류는 일체 반환을 하지 않음
4. 제출 파일명 작성 요령
  - 파일명: [2025 IA x AI 해커톤]\_팀명
5. 서체: 맑은고딕, 크기: 12p, 줄간격: 160%
6. 제출처: 깃허브에 업로드

## 「2025 IA x AI 해커톤」

| 프로젝트명       | BookToss                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 프로젝트<br>목표  | <ol style="list-style-type: none"> <li><b>문제 인식:</b> 공공 도서관에서 대출 가능 여부를 일일이 확인해야하고, 도서관 위치는 따로 검색해야는 '번거로운 독서 경험'을 '즐거운 발견'으로 혁신</li> <li><b>목표:</b> 시민들에게 “지금 바로 빌릴 수 있는 가장 가까운 도서관”만 깔끔하게 제공. API가 없는 상황에서도 AI 에이전트 기반 웹 크롤링으로 실시간 정보를 수집하고 분석.</li> <li><b>해커톤 MVP:</b> 사용자의 위치와 책 제목을 입력받아 강남, 서초, 송파구의 85개 도서관의 실시간 데이터를 통합 검색하고, 대출 가능 여부와 거리정보를 지도와 함께 표시.</li> </ol>                                                                                                                                                                                                                                                                                                                                                               |
| 개발 환경       | <ol style="list-style-type: none"> <li>프론트엔드: Streamlit — 웹 UI 개발</li> <li>백엔드 파이프라인: LangGraph — 데이터 처리 워크플로우</li> <li>브라우저 자동화: browser-use + Playwright — 크롤링</li> <li>HTML 파싱: BeautifulSoup4 — 도서 정보 추출</li> <li>LLM: GPT-4o-mini (OpenAI) — 동적 사이트 제어</li> <li>지도 API: Kakao Map JavaScript API — 위치 시각화</li> <li>언어: Python — 전체 시스템 구현</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                  |
| 구현 기능       | <ol style="list-style-type: none"> <li>백엔드 파이프라인(LangGraph) <ul style="list-style-type: none"> <li>지역 코드 기반 도서관 포털 URL 매핑</li> <li>에이전트(브라우저 자동화)로 도서 검색 및 HTML 수집</li> <li>다중 페이지 자동 처리, 데이터 추출해 JSONL 저장</li> </ul> </li> <li>프론트엔드 (Streamlit) <ul style="list-style-type: none"> <li>주소 입력 기반 좌표 변환 (Kakao Geocoding API)</li> <li>사용자 위치 기준 도서관 거리순으로 정렬</li> <li>대출 가능 여부 실시간 표시</li> <li>도서관별 상세 정보 시각화 (책 표지, 저자 등)</li> </ul> </li> <li>배포 (Infra / DevOps) <ul style="list-style-type: none"> <li>Nginx Reverse Proxy 구성/ Oracle Cloud 기반 Streamlit 앱 배포</li> <li>도메인 연결: booktoss.bit-habit.com (AWS Route 53)</li> <li>와일드카드 SSL 인증서 적용 (Let's Encrypt)</li> <li>개발 환경 관리 (Streamlit 및 Python 기반 환경 구축)</li> </ul> </li> </ol> |
| 코드<br>주요 설명 | <ol style="list-style-type: none"> <li>LangGraph 파이프라인 구성 (pipeline_graph.py) <pre>def build_graph():</pre> </li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              | <pre>graph = StateGraph(dict) # 상태: 단순 dict graph.add_node("get_library_portal", get_library_portal) graph.add_node("search_book", search_book) graph.add_node("parse_html", parse_html)  graph.set_entry_point("get_library_portal") graph.add_edge("get_library_portal", "search_book") graph.add_edge("search_book", "parse_html") graph.add_edge("parse_html", END) return graph.compile()</pre>                                                                      |
| <b>개발 내용</b>                 | <ol style="list-style-type: none"> <li>1. 통합 파이프라인 구축 LangGraph 기반 3단계 처리: URL 조회 → 검색 자동화 → HTML 파싱</li> <li>2. LLM + 브라우저 자동화: GPT-4o-mini가 Playwright를 통해 도서관 사이트를 직접 조작하여 실시간 데이터 수집</li> <li>3. 멀티 페이지 수집 &amp; 스마트 파싱: JavaScript 실행으로 최대 10페이지 HTML 수집 + BeautifulSoup 기반 통합 파서 적용</li> <li>4. 위치 기반 추천 및 시각화: 사용자-도서관 거리 계산, 대출 상태 시각화, Kakao Map 연동으로 직관적 정보 제공</li> <li>5. 사용자 친화적 UI &amp; 저장 구조: Streamlit 인터페이스로 즉시 검색 지원 + 날짜별 raw/parsed 데이터 저장</li> </ol> |
| <b>시연 영상</b>                 | 제작 완료                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>실행 파일</b>                 | 환경변수 파일을 BookToss.env 로 구글 폼으로 제출 했습니다.                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>기타<br/>(팀 구성,<br/>협업)</b> | <ol style="list-style-type: none"> <li>1. 홍정민:<br/>LangGraph 기반 백엔드 파이프라인 및 전체 시스템 아키텍처 설계. 브라우저 자동화 및 데이터 수집을 위한 LLM 에이전트 구현 담당</li> <li>2. 이기찬:<br/>Oracle Cloud 기반 배포 환경 구성 및 Nginx Reverse Proxy 구축. 도메인, SSL 인증서, 배포 자동화 등을 포함한 Infra/DevOps 책임</li> <li>3. 이수민:<br/>Streamlit 기반 웹 프론트엔드 개발 및 사용자 UX/UI 설계 공공도서관 서비스 분석을 위한 리서치 수행</li> </ol>                                                                                                                       |



## 붙임1 붙임

### 1. 프로젝트 구조

```
brower-use-library/  
├─ app.py # 프론트엔드  
(Streamlit UI)  
├─ 00_src/  
│   ├── configs/  
│   │   └─ catalog_index.yaml # 도서관 URL 설정  
│   ├── nodes/  
│   │   ├── get_library_portal.py # 노드 1  
│   │   ├── search_book.py # 노드 2  
│   │   └─ parse_html.py # 노드 3  
│   ├── graph/  
│   │   └─ pipeline_graph.py # LangGraph 파이프라인  
│   └─ data/  
│       ├── raw/ # HTML 원본 (날짜별)  
│       └─ parsed/ # JSONL 결과 (날짜별)  
└─ requirements_freeze.txt # 패키지 버전 고정
```

### 2. 데이터 흐름도

[사용자]

주소: "서울특별시 강남구 역삼동"  
책: "숨결이 바람 될 때"

↓

app.py (Streamlit)

- Kakao API: 주소 → 좌표 (37.5, 127.0)
- 지역 확인: "강남구" → place="gangnam"
- 파이프라인 실행: run\_once(place, title)

↓

pipeline\_graph.py (LangGraph)

Node 1 → Node 2 → Node 3 → END

↓            ↓            ↓  
URL        HTML        JSONL

↓

|      |      |    |
|------|------|----|
| YAML | 브라우저 | 파싱 |
| 설정   | 자동화  | 추출 |

↓

00\_src/data/parsed/2025-11-01/gangnam\_results.jsonl  
{  
 "title": "숨결이...",  
 "library": "행복한도서관",  
 "status\_raw": "대출가능",  
 "available": true,  
 ...  
}

↓

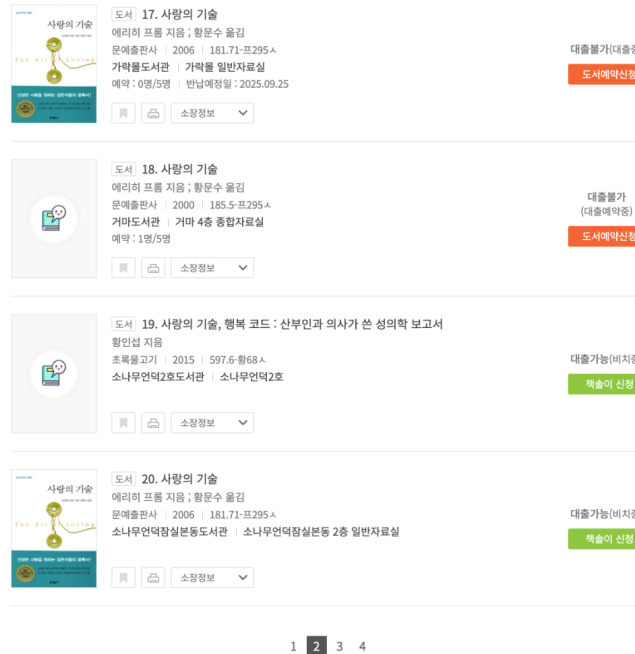
app.py (결과 처리)

- JSONL 읽기
- 도서관별 거리 계산
- 가까운 순 정렬
- Kakao Map에 마커 표시

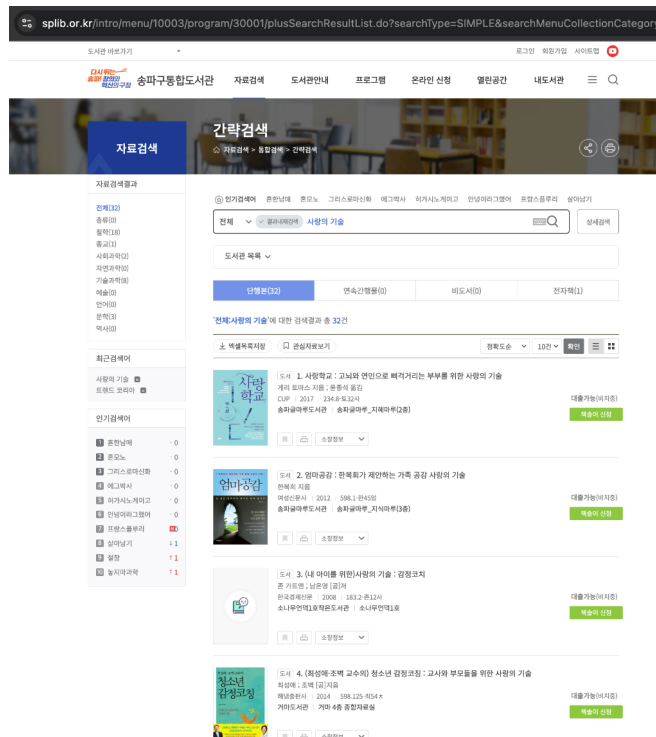
↓

[사용자에게 결과 표시]

### 3. 불편한 기존 공공 도서관 UI/UX



- 원하는 책을 검색했을 때 관련 없는 책들이 상위에 표시됨. 원하는 책은 3페이지 중간쯤에 표시됨.
- 대출 불가와 대출 가능한 책들이 섞여서 표시됨.



- 대출 가능한 책들만 볼 수 있는 필터링 기능이 없어서 불편함
- 검색 후에는 도서관 위치가 나오지 않아서 사용자가 일일이 지도 앱을 별도로 켜서 거리 검색을 해야하는 번거로움이 있음.

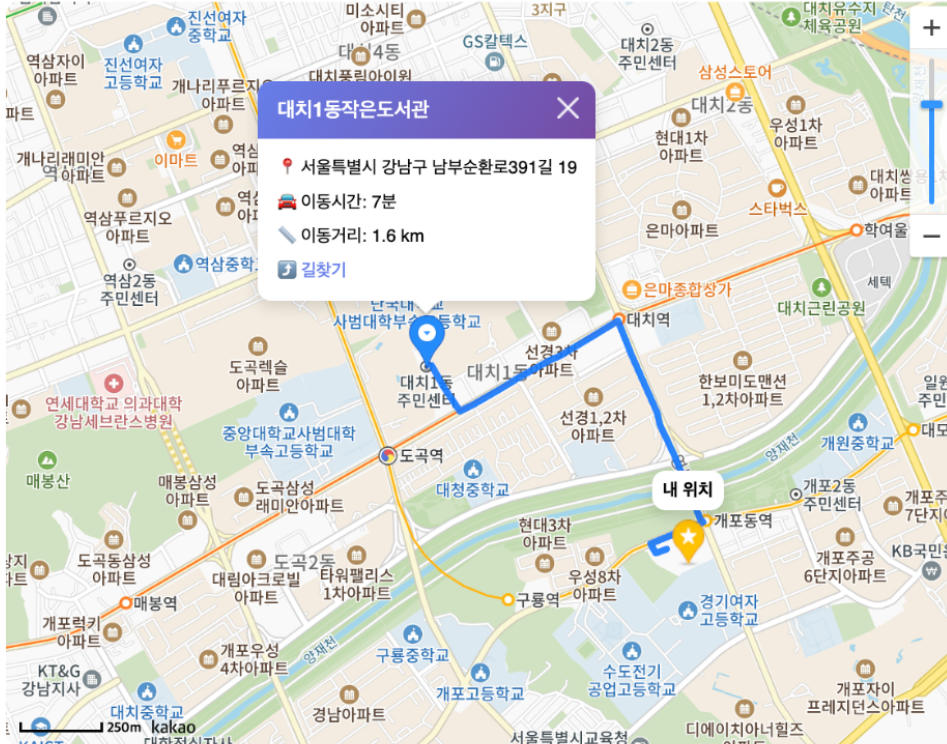


- 도서관 홈페이지에 접속하면 팝업과 지자체 홍보물로 사용자의 혼란을 유발.

#### 4. BookToss 구현



## 가장 가까운 도서관



### 1 대치1동작은도서관 차로 7분

서울특별시 강남구 남부순환로391길 19 [길찾기](#)

#### ☑ 대출 가능 도서 1권



#### 두더지의 고민

- 저자: 김상근
- 청구기호: 유 813.8-김52두



## 1 대치1동작은도서관 차로 7분

서울특별시 강남구 남부순환로391길 19 [길찾기](#)

### ✓ 대출 가능 도서 1권



#### 두더지의 고민

· 저자: 김상근  
· 청구기호: 유 813.8-김52두

## 2 개포하늘꿈도서관 차로 9분

서울특별시 강남구 개포로110길 54 [길찾기](#)

### ✓ 대출 가능 도서 1권



#### 두더지의 고민

· 저자: 김상근  
· 청구기호: 유 813.8-김52두

## 대치도서관 차로 10분

서울특별시 강남구 삼성로 212 [길찾기](#)

### ✓ 대출 가능 도서 1권



#### 두더지의 고민

· 저자: 김상근 글,  
· 청구기호: JM 813.8-김52드